# Hackatón de Estudio

## Camino a la Certificación Full Stack Python

---

**Actividad:** Esta actividad es un **"Hackathon de Estudio"** diseñado para que naveguen a profundidad por nuestro PDF oficial (*MANUAL Curso Intensivo de Python 3ra Edición*).

**Instructor:** Felipe Cuevas

# Índice


1 **Introducción a la Hackatón**
Explicación de la actividad

2 **Etapa I: Laboratorio de Refactorización**
Funciones y Módulos

3 **Etapa II: Safari de Objetos**
POO y Herencia

4 **Etapa III: Simulacro de Desastre**
Archivos y Excepciones

5 **Etapa IV: Radiografía del Framework**
Arquitectura Django

6 **Etapa V: El Arquitecto Web**
Integración Final en Django

7 **Entregable Final**
Descripción del entregable que consolide cada una de las Etapas + Entregables

8 **Conclusión**
Cierre de la actividad y objetivos próximos

# Introducción a la Hackatón

## El Arte de Investigar

¡Hola, futuros desarrolladores Full Stack!

Llegar a esta etapa del bootcamp es un logro gigante, y están a un solo paso de enfrentarse a su **examen de certificación de Talento Digital para Chile**. Sabemos que han escrito mucho código, han resuelto errores frustrantes y han visto cómo sus ideas cobran vida en el navegador. Sin embargo, hay una habilidad invisible que separa a un programador junior de un verdadero profesional: **la capacidad de devorar documentación técnica y encontrar respuestas por su cuenta.**

> Nadie se memoriza un manual de 500 páginas, ¡ni siquiera los desarrolladores senior! Lo que sí hacen es saber *dónde* buscar, *cómo* leer la información técnica y *cómo* aplicarla de inmediato a sus proyectos.

**¿Qué haremos hoy?** En esta clase dejaremos de lado los tutoriales guiados. Hoy se convertirán en investigadores técnicos. Esta actividad es un **"Hackathon de Estudio"** diseñado para que naveguen a profundidad por nuestro PDF oficial (*MANUAL Curso Intensivo de Python 3ra Edición*).

Recorrerán diferentes "estaciones" o etapas. En cada una, se les presentará un problema del mundo real.

**Su misión:** Bucear en el manual, encontrar la teoría que lo resuelve, debatir con sus compañeros y documentar sus hallazgos.

¡Prepárense un buen café, abran el manual y comencemos a entrenar para esa certificación!

Su instructor **Felipe Cuevas**.

ETAPA I

# Laboratorio de Refactorización

## *Funciones y Módulos*

### Objetivo de la Etapa

Comprender cómo Python nos permite crear funciones extremadamente flexibles que aceptan cantidades desconocidas de datos, y aprender a organizar nuestro proyecto en múltiples archivos para mantener un código limpio y profesional.

### Contexto: "Los Limpiadores de Código"

Imagina que te contratan en una startup y te entregan un archivo de Python que tiene 3,000 líneas de código. Todo está en un solo archivo inmenso y las funciones son rígidas (si una función pide 3 datos y le envías 4, el programa explota). Tu primera tarea como desarrollador estrella es "refactorizar" este desastre: necesitas flexibilizar las funciones para que no colapsen si reciben más información de la esperada, y debes dividir ese archivo gigante en módulos más pequeños y ordenados.

---

## Misión Investigativa

Abre tu manual y sumérgete en el capítulo de **Funciones**.

Necesitas descubrir los secretos de los argumentos arbitrarios y las reglas de importación de módulos.

PREGUNTAS Y ENTREGABLE

# Preguntas de la Etapa I

## 1

### El misterio de los asteriscos

A veces no sabemos cuántos datos nos enviará un usuario. Según el manual, ¿cuál es la diferencia exacta y práctica entre usar un asterisco (`*args`) y dos asteriscos (`**kwargs`) en los parámetros de una función? Además, especifiquen qué tipo de estructura de datos (lista, tupla, diccionario, etc.) genera internamente Python con cada uno.

## 2

### El tráfico de importaciones

El manual menciona varias formas de traer código de un archivo a otro (por ejemplo: `import modulo` versus `from modulo import funcion as f`). Construyan un breve cuadro comparativo o lista indicando en qué escenario de la vida real es obligatorio o recomendable usar cada una de estas estrategias para evitar colisiones (que dos funciones se llamen igual y el programa se confunda).

## Entregable de la Etapa I

En su documento de respuestas, deberán redactar lo siguiente:

- Incluyan sus respuestas a las 2 preguntas planteadas: **El misterio de los asteriscos** y **El tráfico de importaciones**.
- Escriban el seudocódigo (o código Python directo) de una función llamada `crear_perfil_usuario`. Esta función debe exigir de manera obligatoria el nombre y el apellido del usuario, pero debe ser capaz de recibir una **cantidad ilimitada de datos extra** (como edad, ciudad, profesión, etc.) sin arrojar error.
- Supongan que guardaron esa función en un archivo llamado `perfiles_app.py`. Escriban la línea de código exacta que usarían en su archivo principal (`main.py`) para importar *solo* esa función, pero asignándole el alias `nuevo_perfil` para usarla más rápido.

ETAPA II

# Safari de Objetos

## *POO y Herencia*

### Objetivo de la Etapa

Dominar la Programación Orientada a Objetos (POO), comprendiendo cómo diseñar moldes inteligentes (clases), darles vida (instanciación) y, sobre todo, reciclar código como verdaderos profesionales mediante la herencia y la composición.

### Contexto: "Los Ingenieros de Sistemas"

Imagina que te han asignado liderar el backend del nuevo portal del colegio (nuestro famoso SGE). Tienes que modelar a los usuarios del sistema. Podrías escribir el código de un Alumno, luego copiar y pegar casi lo mismo para un Profesor, y volver a copiar para el Director. ¡Pero eso es una pesadilla de mantenimiento! Un verdadero desarrollador crea un "molde base" (una clase Usuario) y luego hace que el resto *herede* ese ADN, modificando solo lo necesario. Hoy actuarán como ingenieros de software diseñando la arquitectura de estas entidades.

---

## Misión Investigativa

Abre tu manual oficial y dirígete al fascinante capítulo de Clases.

Su misión es rastrear cómo Python construye objetos, cómo se transfiere la información de una clase padre a una clase hija, y cómo podemos armar objetos complejos como si fueran piezas de Lego.

PREGUNTAS Y ENTREGABLE

# Preguntas de la Etapa II

**1**

### El ADN del objeto

En casi todas las clases que verán en el manual hay un método especial llamado `__init__()`. Investiguen y expliquen con sus propias palabras: ¿Qué función vital cumple este método exactamente en el momento en que "nace" un objeto? Además, ¿por qué el parámetro `self` debe ir en el primer lugar de los paréntesis?

**2**

### El puente generacional

Cuando una clase hija hereda de una clase padre (por ejemplo, un `CocheElectrico` que hereda de `Coche`), solemos ver la función `super().__init__()` en su interior. ¿Qué hace exactamente esta línea mágica y por qué nuestro programa fallaría si la omitimos al intentar usar propiedades de la clase padre?

**3**

### El debate arquitectónico (Herencia vs. Composición)

El manual hace una distinción brillante al explicar que no todo debe ser herencia. A veces es mejor extraer características a una clase separada (como separar la `Bateria` del `CocheElectrico`). Busquen esta sección y debatan: ¿En qué escenario del mundo real elegirían usar "Composición" (tener un objeto dentro de otro) en lugar de "Herencia" (ser un subtipo de otro)? Den 1 ejemplo propio.

## Entregable de la Etapa II

En su documento de respuestas, deberán dibujar o estructurar (puede ser con viñetas, un esquema de cajas en texto, o una foto de un dibujo a mano alzada) la arquitectura de usuarios del sistema:

- Creen una jerarquía conceptual donde exista una clase padre llamada `Usuario` (con atributos base como nombre y correo).
- De ella, debe heredar una clase hija llamada `Administrador`.
- Finalmente, apliquen el concepto de **Composición**: Creen una clase completamente separada e independiente llamada `Privilegios` (que contenga una lista de permisos como "borrar_usuarios", "editar_cursos"). Muestren en su esquema cómo esta clase `Privilegios` se conectaría *dentro* del `Administrador` sin usar herencia.
- También deben incluir sus respuestas a las 3 preguntas planteadas: **El ADN del objeto, El puente generacional** & **El debate arquitectónico (Herencia vs. Composición).**

ETAPA III

# Simulacro de Desastre

## *Archivos y Excepciones*

### Objetivo de la Etapa

Aprender a crear aplicaciones "a prueba de balas" que no colapsen ante errores inesperados (del usuario o del sistema operativo) y dominar la persistencia de datos para que la información no se evapore al apagar el servidor.

### Contexto: "El Escuadrón de Emergencias"

Imaginen este escenario: Es viernes por la tarde, acabamos de lanzar la primera versión de nuestro Sistema de Gestión Escolar (SGE) y está en vivo. De repente, un secretario intenta cargar una lista de alumnos desde un archivo que ya fue borrado del disco duro, y otro usuario escribe la palabra "veinte" en lugar del número "20" en el campo de edad.

Si el código no está preparado, el programa mostrará un aterrador bloque de texto rojo (Traceback) y el sistema entero colapsará. ¡Un desastre total! Hoy actuarán como el equipo de respuesta a incidentes. Su trabajo es blindar la aplicación, anticipar el caos y asegurar que el sistema maneje las crisis con elegancia sin detenerse jamás.

---

## Misión Investigativa

Abran su manual oficial y diríjanse al capítulo de **Archivos y Excepciones**.

Su misión es descubrir las técnicas de programación defensiva para "atrapar" errores en el aire y explorar cómo guardar la memoria de su programa a largo plazo.

PREGUNTAS Y ENTREGABLE

# Preguntas de la Etapa III

1

### El guardián de los archivos

Leer o escribir archivos en el disco duro es una operación delicada. Según el manual, ¿qué ventaja crítica y de seguridad nos ofrece abrir un archivo usando la declaración `with open(...)` en lugar de simplemente usar `open()` y acordarnos de usar `close()` manualmente al final?

2

### La anatomía del rescate (try-except-else)

Atrapar errores es un arte. Investiguen esta estructura y respondan: ¿Por qué es considerada una terrible práctica poner *absolutamente todo* nuestro código dentro del bloque `try`? Además, ¿qué tipo de código o instrucciones deben ir específicamente alojadas en el bloque `else`?

3

### Memoria a largo plazo (JSON)

Si queremos guardar las preferencias de un usuario para la próxima vez que inicie sesión, ¿por qué el manual recomienda enfáticamente utilizar el módulo `json` (`json.dump` / `json.load`) en lugar de simplemente guardar la información como texto plano convencional usando `write()`?

## Entregable de la Etapa III

En su documento de respuestas, deberán redactar un **"Protocolo de Emergencia"** en código Python.

- Escriban un bloque lógico utilizando `try-except` que intente cargar la información de un archivo llamado `configuracion_usuarios.json`.
- Si el archivo no existe en el sistema (es decir, ocurre la excepción `FileNotFoundError`), el programa no debe colapsar. Deben aplicar la técnica de **"fallar silenciosamente"** utilizando la instrucción `pass` (o alternativamente, mostrar un mensaje amigable indicando que se usarán las configuraciones por defecto).
- Asegúrense de dejar comentarios en su código explicando qué hace cada bloque.
- También deben incluir sus respuestas a las 3 preguntas planteadas: **El guardián de los archivos, La anatomía del rescate (try-except-else)** & **Memoria a largo plazo (JSON).**

ETAPA IV

# Radiografía del Framework

## *Arquitectura Django*

### Objetivo de la Etapa

Desmitificar la magia detrás de una aplicación web, comprendiendo a nivel estructural cómo funciona el patrón MVT (Modelo-Vista-Plantilla) de Django y cómo se comunican las distintas partes del código para que una página aparezca en el navegador.

### Contexto: "Los Arquitectos de la Web"

Hasta ahora, hemos construido scripts que viven y mueren en la oscura pantalla de la consola (terminal). Pero el mundo real funciona en navegadores, con botones, colores y usuarios concurrentes. Imaginen que nuestro SGE (Sistema de Gestión Escolar) acaba de recibir su primera petición web: un apoderado escribió la dirección `www.colegio.cl/notas/` y apretó "Enter".

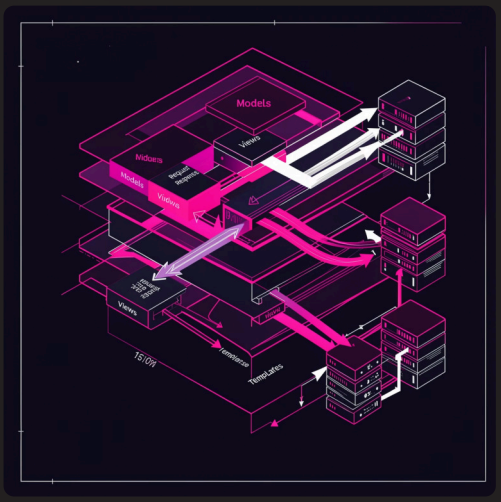
¿Qué pasa exactamente en los milisegundos siguientes? Hoy actuarán como Arquitectos de Software. Su misión no es escribir código, sino trazar el mapa interno de Django. Necesitan entender cómo viaja la información desde que el usuario hace clic hasta que el servidor le devuelve la página construida. ¡Si entienden este mapa, podrán rastrear y solucionar el 90% de los errores web!

---

## Misión Investigativa

Abran su manual oficial y viajen a la **Tercera Parte: Aplicaciones Web** (donde se construye el proyecto *Learning Log*).

Su misión es mapear el esqueleto de Django y el famoso ciclo de "Petición/Respuesta" (Request/Response).



PREGUNTAS Y ENTREGABLE

# Preguntas de la Etapa IV

1

### La anatomía del sistema

Al iniciar un desarrollo en Django, el manual hace una distinción estricta entre crear un "Proyecto" y crear una "Aplicación". Investiguen y expliquen con sus propias palabras: ¿Cuál es la diferencia técnica y organizativa entre ambos conceptos?

2

### La sala de control (Migraciones)

Hemos diseñado nuestros modelos de datos (clases de Python) en la Etapa 2. Sin embargo, SQLite no entiende de clases, solo entiende de tablas. Según el manual, ¿cuáles son los comandos exactos de consola (terminal) que debemos ejecutar secuencialmente para que Django traduzca nuestro archivo `models.py` y construya las tablas reales en la base de datos?

3

### El semáforo y el recepcionista

Describan el flujo de trabajo de Django. Cuando un usuario ingresa a una URL en su navegador: ¿Qué archivo actúa como el "semáforo" para saber a dónde ir? ¿Qué archivo actúa como el "recepcionista" procesando la lógica? ¿Y quién se encarga de pintar el resultado final? (Pista: investiguen la relación entre `urls.py`, `views.py` y las plantillas).

## Entregable de la Etapa IV

En su documento de respuestas, deberán construir un diagrama de flujo o mapa mental secuencial titulado **"El Viaje de un Request"**.

- El viaje debe comenzar con el evento: *"El usuario entra a* `http://localhost:8000/alumnos/` *".*
- Debe ilustrar paso a paso por cuáles archivos pasa esa petición web (`urls.py` → `views.py` → `models.py` [si necesita datos] → template HTML).
- Pueden hacerlo mediante un listado enumerado claro, un dibujo adjunto, o utilizando herramientas de diagramado. Lo importante es que si alguien ajeno al bootcamp lee ese entregable, entienda perfectamente cómo funciona el esqueleto de Django.
- También deben incluir sus respuestas a las 3 preguntas planteadas: **La anatomía del sistema, La sala de control (Migraciones)** & **El semáforo y el recepcionista.**

ETAPA V

# El Arquitecto Web

## *Integración Final en Django*

### Objetivo de la Etapa

Conectar todas las piezas del rompecabezas: unir la base de datos, la lógica del servidor y las interfaces HTML utilizando el poder del ORM y el motor de plantillas de Django para crear páginas web 100% dinámicas.

### Contexto: "Los Maestros Constructores"

¡Llegamos a la prueba final! Su equipo de diseño frontend les acaba de entregar un archivo HTML hermoso con la tabla de "Lista de Alumnos" para nuestro SGE. El problema es que los nombres de los alumnos en esa tabla son falsos (están escritos directamente en el código HTML a mano).

Como Maestros Constructores, su trabajo es "inyectarle vida" a esa plantilla. Tienen que ir a la base de datos, extraer a los alumnos reales usando Python y enviarlos al HTML para que la página se construya sola. Esta es la esencia del desarrollo Full Stack: hacer que lo estático se vuelva dinámico.

---

## Misión Investigativa

Abran su manual oficial y sumérjanse en las secciones donde se construyen las **Vistas (views) y Plantillas (templates)** del proyecto *Learning Log*.

Necesitan descifrar el lenguaje secreto que permite que el HTML entienda las variables de Python.

PREGUNTAS Y ENTREGABLE

# Preguntas de la Etapa V

1

### La identidad de los datos

En el archivo `models.py`, casi siempre definimos un método especial llamado `__str__()` dentro de nuestras clases. Según el manual, ¿para qué sirve exactamente este método y cómo afecta visualmente la forma en que administramos nuestros datos en el panel de control (admin) de Django?

2

### El lenguaje de las plantillas

HTML no entiende de código Python nativo. Django inventó una sintaxis especial para inyectar datos en el HTML. Investiguen y expliquen: ¿Cuál es la diferencia estricta entre usar llaves dobles `{{ }}` y usar llaves con porcentajes `{% %}`? Den un ejemplo de cómo usarían esta sintaxis para hacer un bucle `for` que imprima los nombres de una lista de alumnos enviada desde la vista.

3

### El esqueleto maestro (Herencia de plantillas)

Imaginen que nuestro SGE tiene 50 páginas y el director nos pide cambiar el color de la barra de navegación en todas ellas. Sería una locura editar 50 archivos HTML. Investiguen el concepto de "Herencia de plantillas" (`base.html`). ¿Por qué esta herramienta de Django es una de las más poderosas para mantener un sitio web gigante escalable y ordenado?

## Entregable de la Etapa V

En su documento de respuestas, deberán redactar un **Mini-Ensayo Técnico (de máximo 3 párrafos)**.

- En este ensayo deben argumentar y defender por qué aprender Programación Orientada a Objetos (la Etapa 2 de nuestra investigación) es un requisito estrictamente obligatorio para poder entender y crear Modelos (`models.py`) en Django.
- Conecten los puntos: expliquen cómo una clase de Python se relaciona con una tabla en la base de datos y cómo los atributos se convierten en columnas. ¡Demuestren que entienden el "por qué" detrás del código!
- También deben incluir sus respuestas a las 3 preguntas planteadas: **La identidad de los datos, El lenguaje de las plantillas** & **El esqueleto maestro (Herencia de plantillas).**

# Entregable Final

## El Sello del Desarrollador Moderno: IA Generativa

---

### Objetivo del Entregable

Un desarrollador Full Stack de alto nivel no solo escribe buen código, sino que sabe comunicarlo. Su última misión en este Hackathon es consolidar todo el conocimiento investigado en las 5 etapas en un formato visual espectacular, demostrando su **maestría en el uso de Inteligencia Artificial (IA)**.

**IMPORTANTE**

Deberán exponer su entregable final ante la clase y su instructor.

### Las Opciones de Consolidación

Elijan **UNA** de las siguientes opciones para presentar su trabajo final. En ambas, el uso de Inteligencia Artificial es obligatorio para acelerar el desarrollo:

#### Opción A - Presentación

- **El reto:** Crear un conjunto de diapositivas de alto impacto visual que resuma sus códigos, diagramas y respuestas clave.
- **Herramientas IA:** ¡Prohibido partir de cero! Utilicen **Gemini** para estructurar los textos, **NotebookLM** para sintetizar el manual y crear presentaciones, o **Gamma** para generar el documento.

#### Opción B - Sitio Web

- **El reto:** Construir un sitio web estático (una sola página HTML) que contenga secciones elegantes para lucir los hallazgos de cada etapa.
- **Herramientas IA:** Utilicen el poder de **Gemini** en combinación con **Google Stitch** para maquetar la estructura y el diseño con Tailwind CSS en tiempo récord.

# Conclusión

## La Mentalidad del Desarrollador

¡Felicidades, equipo! Si llegaron hasta aquí y completaron sus entregables, acaban de sobrevivir a su primer Hackathon de Estudio intensivo. Sé perfectamente que leer documentación técnica, debatir conceptos y bucear en un manual de cientos de páginas puede no parecer tan glamuroso como teclear líneas de código sin parar. Pero créanme: hoy acaban de dar el salto más importante en su carrera profesional.

> Allá afuera, en el mundo real de la industria tecnológica, las librerías se actualizan, las versiones cambian y las herramientas evolucionan. Un programador amateur copia y pega código de foros o de una inteligencia artificial hasta que el programa mágicamente funciona; pero **un desarrollador Full Stack profesional entiende el *por qué***.

## La Mirada en la Certificación

El examen de certificación de Talento Digital para Chile que tienen en el horizonte no busca evaluar si son diccionarios humanos que memorizaron sintaxis. Lo que busca evaluar es exactamente lo que entrenaron hoy: su capacidad analítica, su solidez conceptual y su habilidad arquitectónica para conectar el backend con el frontend resolviendo problemas reales.

## Próximos Pasos

Ahora, tómense un momento para respirar. Revisen sus apuntes, comparen sus diagramas y prepárense para defender sus descubrimientos en nuestra mesa redonda. Siéntanse profundamente orgullosos del nivel técnico y analítico que han alcanzado.

- Han dejado de ser solo estudiantes que siguen tutoriales para convertirse en verdaderos investigadores del código. ¡La arquitectura de sus proyectos es sólida, y esa certificación ya casi está en sus manos!